

Session 5: Interfaces in C#

In this session, we will cover the following topics related to interfaces in C#:

1. **Implementing an Interface**
2. **Explicitly Implementing an Interface**
3. **Inheritance in Interfaces**
4. **Default Interface Methods**
5. **Operator Overloading**

Let's dive into each topic in detail.

1. Implementing an Interface

What is an Interface?

- An **interface** is a contract that defines a set of methods, properties, events, or indexers that a class or struct must implement.
- Interfaces cannot contain any implementation; they only provide the signatures of the members.
- By convention, interface names in C# start with the letter "I".

Defining an Interface

To define an interface, you use the `interface` keyword. Here's a simple example:

```
csharp
VerifyOpen In EditorRunCopy code
1public interface IAnimal
2{
3    string Name { get; set; }
4    void MakeSound();
5}
```

Implementing an Interface

A class implements an interface using the `:` symbol. The class must provide implementations for all members of the interface. Here's how you can implement the `IAnimal` interface:

```
csharp
VerifyOpen In EditorRunCopy code
1public class Dog : IAnimal
2{
3    public string Name { get; set; }
4
5    public Dog(string name)
6    {
7        Name = name;
8    }
9
10   public void MakeSound()
11   {
12       Console.WriteLine("Woof! Woof!");
13   }
14}
15
```

```

16public class Cat : IAnimal
17{
18    public string Name { get; set; }
19
20    public Cat(string name)
21    {
22        Name = name;
23    }
24
25    public void MakeSound()
26    {
27        Console.WriteLine("Meow!");
28    }
29}

```

Using the Interface

You can use the interface type to refer to objects of the implementing classes, allowing for polymorphism:

```

csharp
VerifyOpen In EditorRunCopy code
1public class Program
2{
3    public static void Main(string[] args)
4    {
5        IAnimal myDog = new Dog("Buddy");
6        IAnimal myCat = new Cat("Whiskers");
7
8        Console.WriteLine($"Dog's name: {myDog.Name}");
9        myDog.MakeSound();
10
11        Console.WriteLine($"Cat's name: {myCat.Name}");
12        myCat.MakeSound();
13    }
14}

```

2. Explicitly Implementing an Interface

Explicit implementation allows you to define how a class implements the members of an interface in a way that is distinct from the class's own members. This is useful when a class implements multiple interfaces that may have members with the same name.

Key Concepts

- **Syntax:** Prefix the member with the interface name followed by a dot.
- **Accessibility:** Explicitly implemented members are not accessible through the class instance directly; they can only be accessed through an instance of the interface.

Example

```

csharp
VerifyOpen In EditorRunCopy code
1interface IEnglishDimensions
2{
3    float Length();
4    float Width();
5}
6
7interface IMetricDimensions

```

```

8{
9    float Length();
10   float Width();
11}
12
13class Box : IEnglishDimensions, IMetricDimensions
14{
15    private float lengthInches;
16    private float widthInches;
17
18    public Box(float lengthInches, float widthInches)
19    {
20        this.lengthInches = lengthInches;
21        this.widthInches = widthInches;
22    }
23
24    float IEnglishDimensions.Length() => lengthInches;
25    float IEnglishDimensions.Width() => widthInches;
26
27    float IMetricDimensions.Length() => lengthInches * 2.54f;
28    float IMetricDimensions.Width() => widthInches * 2.54f;
29}

```

3. Inheritance in Interfaces

Interfaces can inherit from other interfaces, allowing for a form of multiple inheritance. A class can implement multiple interfaces, promoting code reuse and separation of concerns.

Example

```

csharp
VerifyOpen In EditorRunCopy code
1public interface IShape
2{
3    double CalculateArea();
4}
5
6public interface IColoredShape : IShape
7{
8    string Color { get; set; }
9}
10
11public class Circle : IColoredShape
12{
13    public double Radius { get; set; }
14    public string Color { get; set; }
15
16    public Circle(double radius, string color)
17    {
18        Radius = radius;
19        Color = color;
20    }
21
22    public double CalculateArea()
23    {
24        return Math.PI * Radius * Radius;
25    }
26}

```

4. Default Interface Methods

Introduced in C# 8.0, default interface methods allow you to add new methods to interfaces without breaking existing implementations.

Key Features

- **Backward Compatibility:** Add new methods with default implementations.
- **Virtual Methods:** Default methods can be overridden by implementing classes.

Example

```
csharp
VerifyOpen In EditorRunCopy code
1public interface ILogger
2{
3    void Log(string message); // Abstract method
4
5    void Log(string message, LogLevel logLevel)
6    {
7        Console.WriteLine($"Log Level: {logLevel}");
8        Console.WriteLine(message);
9    }
10}
```

5. Operator Overloading

Operator overloading allows you to define custom behavior for operators (like +, -, *, etc.) for your classes. This can make your classes easier to use and more intuitive.

Example

```
csharp
VerifyOpen In EditorRunCopy code
1public class Point
2{
3    public int X { get; set; }
4    public int Y { get; set; }
5
6    public Point(int x, int y)
7    {
8        X = x;
9        Y = y;
10   }
11
12   public static Point operator +(Point p1, Point p2)
13   {
14       return new Point(p1.X + p2.X, p1.Y + p2.Y);
15   }
16}
```

Conclusion

Understanding interfaces in C# is crucial for designing flexible and maintainable applications. They promote loose coupling, allow for polymorphism, and enable multiple inheritance of behavior. Default interface methods and explicit implementations further enhance the usability and functionality of interfaces. Operator overloading adds another layer of expressiveness to your classes, making them more intuitive to work with.

